Control 2 CC4303
Parte 1 — Sockets orientados a conexión con Stop & Wait

Integrante Julián Proboste

Introducción

En esta actividad se implementó la clase `SocketTCP`, que entrega sockets orientados a conexión usando exclusivamente sockets UDP como transporte. Sobre ella se construyeron un cliente y un servidor capaces de transferir un archivo de origen a destino de manera íntegra, incluso cuando la red induce pérdida de segmentos y retardo en el envío.

El protocolo sigue el esquema de Stop & Wait visto en clases. Antes de enviar datos las dos partes establecen la conexión con un 3-way handshake, durante la transferencia el emisor envía un segmento y espera su ACK antes de pasar al siguiente retransmitiendo en caso de timeout, y al finalizar se libera la conexión con una secuencia de cierre basada en FIN y ACK. El estado completo de la conexión, direcciones, números de secuencia, buffers y timeouts, vive dentro de la clase y su constructor no recibe parámetros.

La implementación usa solo las librerías estándar `socket`, `random`, `sys` y `time`. Las pérdidas se indujeron con netem sobre la interfaz `lo` de localhost, con 20% de pérdida y 0.5 segundos de retardo. A lo largo de este informe se documentan las decisiones de diseño de cada paso del enunciado, los dos diagramas solicitados y las pruebas finales.

Paso 1. Cliente y servidor provisorios

El primer paso fue un cliente y un servidor provisorios que se comunican directamente con sockets UDP, sin confiabilidad todavía. El cliente lee un archivo desde entrada estándar, lo trocea en pedazos de a lo más 16 bytes y los envía al servidor, el servidor recibe los datagramas y reescribe el contenido en salida estándar.

Este código no forma parte de la entrega, pero fijó dos decisiones que se mantuvieron en el resto de la actividad, el tamaño máximo de payload de 16 bytes por segmento y cargar el archivo completo en memoria como arreglo de bytes antes de trocearlo. También se usó para comprobar con netem que sin confiabilidad los segmentos efectivamente se pierden.

Paso 2-3. Clase SocketTCP y headers

El constructor no recibe parámetros y deja listos todos los recursos que la conexión va a necesitar, el socket UDP, la dirección local y la de destino, el número de secuencia, el timeout fijo, la bandera de debug y el estado de recepción.

```
def __init__(self):
    self.udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

```
    self.local_addr = None
    self.dest_addr = None
    self.seq = 0
    self.timeout = 2
    self.debug = False
    self.message_length = 0
    self.pending = 0
    self.buffer = b""
```

Cada segmento se serializa con un header de 7 bytes, tres bytes de banderas y cuatro de número de secuencia. Cada bandera, `syn`, `ack` y `fin`, ocupa un byte entero con valor 0 o 1, y el `seq` se guarda en formato big-endian. Detrás del header viene el área de datos con a lo más 16 bytes de payload, de modo que el datagrama UDP completo nunca supera los 23 bytes.

La estructura de datos elegida para representar un segmento es un diccionario con las claves `syn`, `ack`, `fin`, `seq` y `data`. Las funciones `parse_segment` y `create_segment` son estáticas porque solo traducen entre bytes y diccionario sin necesitar estado de la instancia.

```
@staticmethod
def parse_segment(segment):
    return {
        "syn": bool(segment[0]),
        "ack": bool(segment[1]),
        "fin": bool(segment[2]),
        "seq": int.from_bytes(segment[3:7], "big"),
        "data": segment[7:],
    }

@staticmethod
def create_segment(parsed):
    return bytes([
        int(parsed["syn"]),
        int(parsed["ack"]),
        int(parsed["fin"]),
    ]) + parsed["seq"].to_bytes(4, "big") + parsed["data"]
```

Un byte por bandera en vez de comprimir las tres en un solo byte costó un poco de tamaño, a cambio el parseo queda legible con simples índices sobre el arreglo, sin usar `struct` ni operaciones bit a bit. El socket UDP siempre se lee y se escribe con un buffer fijo de `HEADER_SIZE + MAX_DATA`, es decir 23 bytes, independiente de `buff_size` que es asunto exclusivo de `recv`.

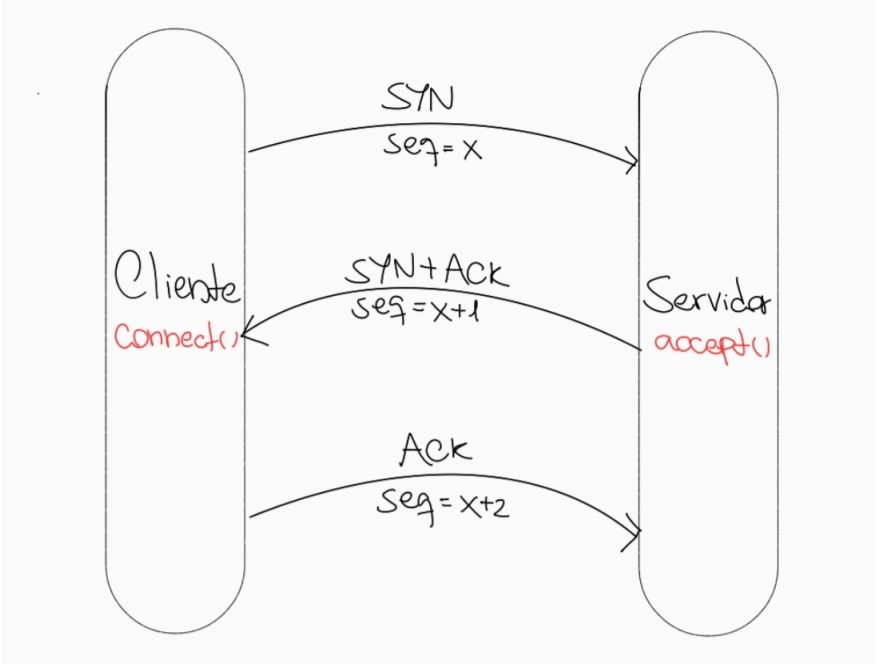
Con esto listo se extendió el código provisorio del paso 1 para que el emisor envuelva cada trozo de datos en un segmento con header y el receptor extraiga el contenido, verificando que los números de secuencia avanzan de 16 en 16 a lo largo del mensaje.

Paso 4. 3-way handshake

`connect` inicia la conexión desde el lado del cliente. Elige un número de secuencia inicial aleatorio entre 0 y 100, envía el SYN con ese seq y queda esperando el SYN+ACK. Al recibirlo responde con el ACK final y la función termina con el estado ya sincronizado.

`accept` espera en el socket de escucha la llegada de un SYN. Cuando recibe uno crea un segundo objeto `SocketTCP` para la conexión y lo asocia a una dirección nueva, distinta de la del socket que llamó a `accept`, con bind en un puerto efímero. La secuencia de ese nuevo socket arranca en el seq del SYN más uno, envía el SYN+ACK y espera el ACK final. Si el handshake se completa retorna el socket de conexión junto a su dirección, y es el que se usa después para enviar y recibir mensajes.

Cada mensaje del handshake avanza el contador en uno, de modo que al terminar ambos lados quedan con el mismo número de secuencia, listo para el primer segmento de datos. Un único contador, `self.seq`, se comparte entre todas las etapas de la conexión.



El diagrama asocia cada mensaje con la función que lo maneja, el SYN y la espera del SYN+ACK corresponden a `connect`, la espera del SYN, la creación del socket nuevo, el SYN+ACK y la espera del ACK final corresponden a `accept`, y el ACK que cierra el handshake corresponde de nuevo a `connect`. La prueba de este paso verificó el handshake completo con los tres mensajes en la terminal.

Paso 5. Stop & Wait

El timeout fijo de la conexión se eligió en 2 segundos. Con netem usando 0.5 segundos de retardo por lado el round trip ronda el segundo, y un timeout de 1 segundo habría disparado retransmisiones espurias que ensucian la salida sin necesidad.

`send` divide el mensaje en partes, la primera con el largo del mensaje en 4 bytes y el resto en trozos de 16 bytes, y procesa cada parte con la misma rutina de Stop & Wait. Guarda el número de secuencia del segmento, calcula el seq que tendrá el ACK esperado, envía y espera. Si el ACK con ese seq llega a tiempo avanza el contador; si se cumple el timeout retransmite el mismo segmento y vuelve a esperar, sin límite de reintentos. El segmento que trae el largo espera su ACK igual que cualquier otro.

```
for part in parts:
    seq_segment = self.seq
    esperado = self.seq + len(part)
    ack_ok = False
    resend = False
    while not ack_ok:
        self._send_segment(False, False, False, seq_segment, part,
resend=resend)
        try:
            while not ack_ok:
                raw, _ = self.udp_socket.recvfrom(HEADER_SIZE + MAX_DATA)
                parsed = self.parse_segment(raw)
                if parsed["ack"] and parsed["seq"] == esperado:
                    ack_ok = True
        except socket.timeout:
            resend = True
    self.seq = esperado
```

`recv` trabaja en dos fases. Cuando `message_length` está en cero el mensaje recién empieza y la primera llamada espera el segmento que trae el largo; a partir de ahí la función acumula segmentos hasta completar el mínimo entre lo que falta del mensaje y `buff_size`, guardado en la variable `target`.

```
target = min(self.pending + len(self.buffer), buff_size)

while len(self.buffer) < target:
    raw, _ = self.udp_socket.recvfrom(HEADER_SIZE + MAX_DATA)
    parsed = self.parse_segment(raw)
    if parsed["seq"] != self.seq or not parsed["data"]:
        self._send_segment(False, True, False, self.seq)
        continue
    self.buffer += parsed["data"]
```

```
    self.pending -= len(parsed["data"])
    self.seq += len(parsed["data"])
    self._send_segment(False, True, False, self.seq)

result = self.buffer[:target]
self.buffer = self.buffer[target:]
```

Las decisiones de este punto son las siguientes. `target` suma lo que aún falta del mensaje con lo que quedó en el buffer de una llamada anterior, de modo que `recv` retorna apenas se cumpla el mínimo pedido por el enunciado y pueda retornar menos que `buff_size` cuando el mensaje se termina. Cuando un segmento trae más bytes de los que faltan para completar `target`, el exceso queda guardado en `self.buffer` y se entrega en la siguiente llamada, así nada se pierde aunque los límites de `buff_size` no caigan en los límites de los segmentos. Un segmento con seq distinto al esperado o sin datos se trata como duplicado, se reenvía el ACK con el seq propio y se descarta, con eso el emisor se entera aunque su ACK original se haya perdido. Cuando `pending` llega a cero y el buffer queda vacío, `message_length` vuelve a cero y la próxima llamada espera un segmento de largo nuevo. Por último, `buff_size` afecta solo a la lógica de `recv`, el socket UDP siempre usa su buffer fijo de 23 bytes.

Paso 6. Cierre de conexión

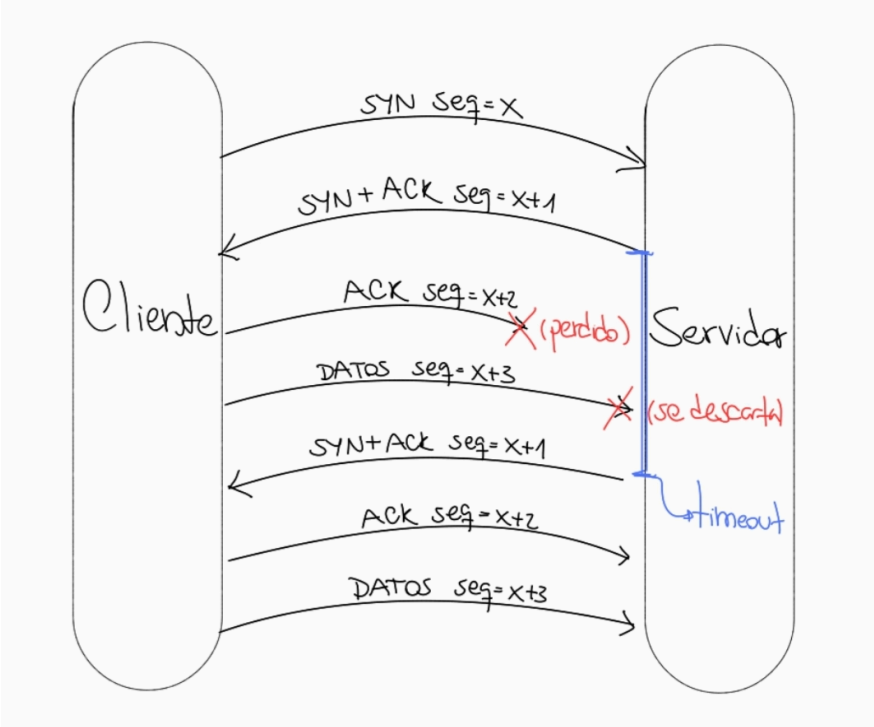
El cierre usa la misma estructura de tres mensajes que el handshake. `close` envía un FIN con el número de secuencia actual, espera el FIN+ACK de la contraparte con el seq siguiente y responde con el ACK final, después libera el socket UDP. `recv_close` hace el lado contrario, espera el FIN, responde con FIN+ACK y espera el ACK final antes de liberar su socket. Los tres mensajes validan el número de secuencia recibido para no confundir segmentos viejos con mensajes del cierre, y al terminar ambos lados el contador compartido quedó avanzado en tres.

En esta versión el cierre todavía no toleraba pérdidas, cualquier timeout en la espera del FIN, del FIN+ACK o del ACK final lanzaba `TimeoutError`. El paso 8 modifica estas dos funciones.

Paso 7. Handshake con pérdidas

`connect` y `accept` pasaron a comportarse con Stop & Wait. Si el SYN o el SYN+ACK se pierde, el que espera cumple su timeout y retransmite el mismo mensaje, sin límite de reintentos, hasta que llega la respuesta esperada.

El caso borde aparece cuando se pierde el último ACK del handshake. El cliente termina `connect` apenas envía ese ACK y pasa directo a `send`, mientras el servidor sigue esperándolo en `accept` y retransmite el SYN+ACK. El cliente recibe entonces un SYN+ACK que ya no le corresponde, y si lo tratara como un segmento más perdería ese mensaje.



La solución sale del diagrama. Si `send` o `recv` reciben un segmento con `syn` y `ack` activos, en lugar de ignorarlo reenvían un ACK con el seq del SYN+ACK recibido más uno, que es exactamente el mensaje que `accept` espera. Como el cliente probablemente ya envió su primer segmento de datos, el diagrama muestra también que ese segmento puede descartarse en `accept` si llega antes que el ACK, sin perder información, porque `send` lo retransmite por timeout y `recv` lo recibe normalmente. En la prueba de este paso, con netem y mensajes largos, la transmisión se completó íntegra hasta el cierre, que era el único punto que todavía no manejaba pérdidas.

Paso 8. Cierre tolerante a pérdidas

`close` pasó a retransmitir el FIN ante cada timeout. Espera hasta 3 timeouts y si al cumplirse el tercero no recibió el FIN+ACK, asume que la contraparte se cerró y libera su socket igual. Si en cambio recibe el FIN+ACK con éxito, reenvía el último ACK tres veces esperando un timeout entre cada envío, para que la probabilidad de que al menos uno llegue sea alta aunque haya pérdida.

`recv_close` espera el FIN hasta 3 timeouts, decisión propia del informe, el enunciado solo pedía límite para el ACK final y sin esto la función se colgaría si todos los FIN se pierden. Al recibir el FIN responde con FIN+ACK y espera el ACK final también hasta 3 timeouts, asumiendo cierre si no llega. Dos situaciones de borde se atienden en estas esperas, un FIN duplicado significa que el emisor no recibió nuestro FIN+ACK, así que se reenvía y se renueva el contador de timeouts, y un segmento de datos duplicado significa que su ACK se perdió, así que se reenvía el ACK correspondiente para que el emisor no quede

retransmitiendo para siempre mientras nosotros esperamos el FIN. Ninguna de las dos funciones lanza excepción, ambas terminan cerrando el socket en todos los caminos, y el modo debug informa los reintentos con líneas como `timeout 1/3 esperando FIN+ACK` y `3 timeouts sin FIN+ACK, asumo que la contraparte cerró`.

Pruebas

Las pruebas se corrieron en localhost con netem induciendo 20% de pérdida y 0.5 segundos de retardo sobre la interfaz `lo`. El comando que activa las pérdidas es el siguiente, y hay que desactivarlo al terminar de probar.

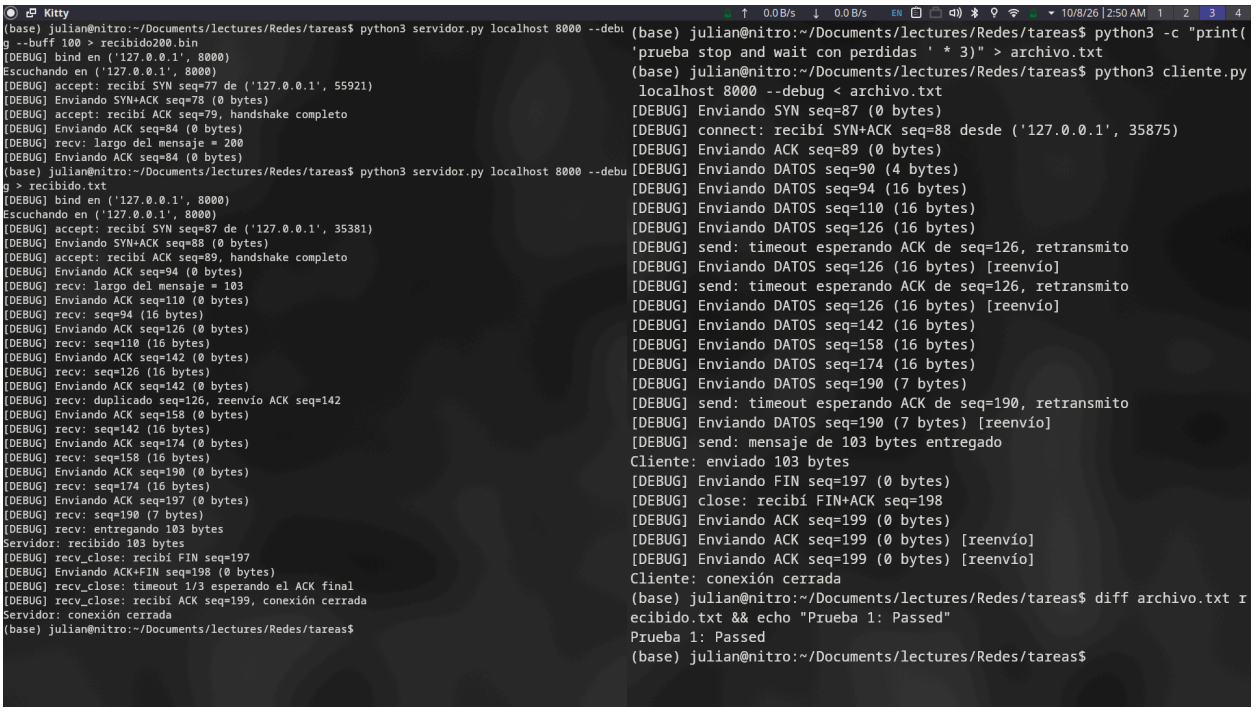
```
sudo tc qdisc add dev lo root netem loss 20.0% delay 0.5s
sudo tc qdisc del dev lo root netem
```

Ambos programas aceptan la bandera `--debug`, que activa el modo debug de la clase. El debug imprime a salida de error cada segmento enviado con su número de secuencia y una marca `[reenvío]` cuando es una retransmisión, junto a los eventos de recepción como segmentos duplicados, timeouts y asunciones de cierre. Como las líneas van por stderr, el archivo que el servidor escribe por stdout queda limpio.

Prueba 1. Transferencia con pérdidas

La primera prueba transfiere `archivo.txt`, de 103 bytes, es decir mayor al payload máximo de 16 bytes, con netem activo. El servidor corre en la terminal izquierda, el cliente en la derecha y el `diff` se ejecuta en la terminal del cliente apenas termina.

```
python3 servidor.py localhost 8000 --debug > recibido.txt
python3 cliente.py localhost 8000 --debug < archivo.txt
diff archivo.txt recibido.txt && echo "Prueba 1: Passed"
```

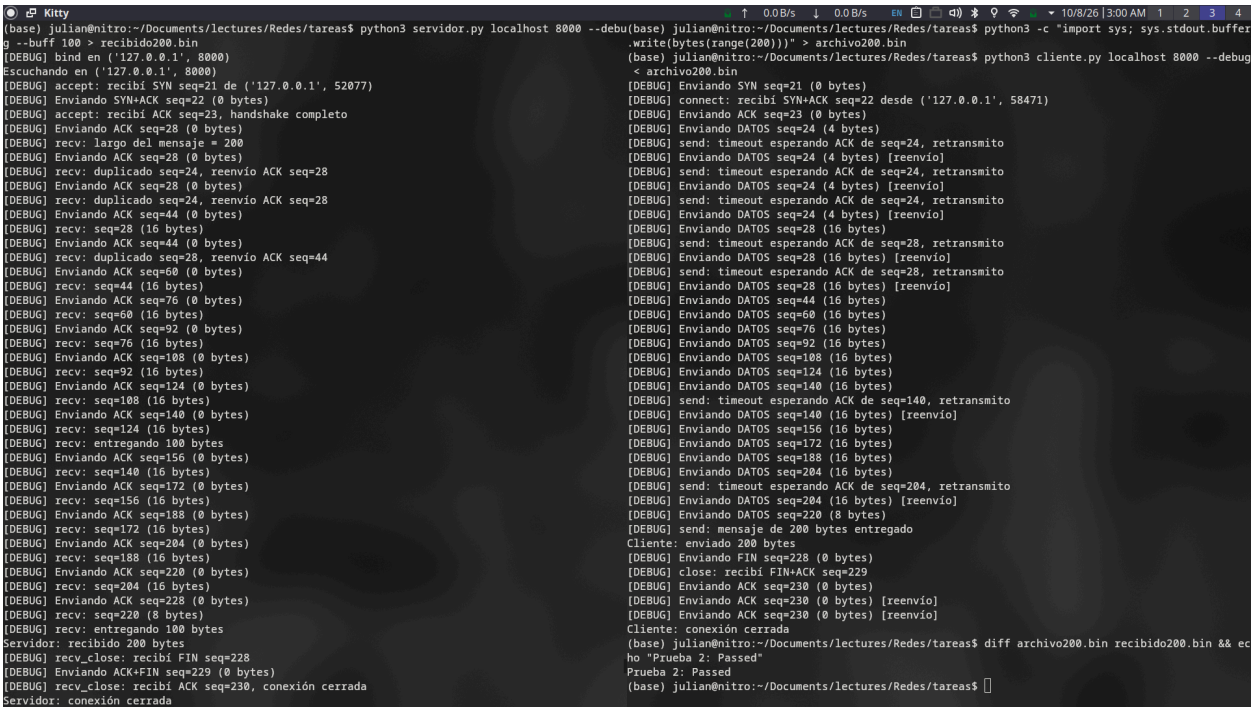


En la captura se ven las retransmisiones de ambos lados. El cliente reenvía segmentos de datos con la marca `[reenvío]` cuando su ACK no llega dentro del timeout, y el servidor responde reenviando el ACK correspondiente cuando recibe un segmento duplicado. El cierre también se recupera de pérdidas, con las líneas de timeout esperando el FIN o el FIN+ACK, el reenvío del FIN, el reenvío del FIN+ACK ante un FIN duplicado y el último ACK saliendo tres veces. Al final el `diff` no reporta diferencias.

Prueba 2. buff_size menor que message_length

La segunda prueba verifica el comportamiento de `recv` cuando `buff_size` es menor que `message_length`. El archivo tiene 200 bytes, es decir 2n con n igual a 100, y 100 no es múltiplo de 16, de modo que los límites de entrega de `recv` no caen en los límites de los segmentos de 16 bytes. El servidor se lanza con `--buff 100` y el loop de recepción hace exactamente dos llamadas a `recv`, una por cada mitad del mensaje.

```
python3 servidor.py localhost 8000 --debug --buff 100 > recibido200.bin
python3 cliente.py localhost 8000 --debug < archivo200.bin
diff archivo200.bin recibido200.bin && echo "Prueba 2: Passed"
```



En la terminal del servidor aparecen las dos líneas `entregando 100 bytes`, una por cada llamada a `recv`, y el mensaje se reconstruye completo a pesar de que ningún límite de `buff_size` coincide con un límite de segmento. El `diff` confirma que los 200 bytes llegaron completos y en orden, y ambos lados cierran la conexión.

Cómo ejecutar el código

La entrega consta de tres archivos, `SocketTCP.py` con la clase, `cliente.py` y `servidor.py`. El servidor se lanza primero y queda escuchando, después se lanza el cliente con el archivo en entrada estándar.

```
# activar pérdidas, opcional
sudo tc qdisc add dev lo root netem loss 20.0% delay 0.5s

# terminal 1
python3 servidor.py localhost 8000 --debug > recibido.txt

# terminal 2
python3 cliente.py localhost 8000 --debug < archivo.txt

# verificación
diff archivo.txt recibido.txt

# apagar las pérdidas
sudo tc qdisc del dev lo root netem
```

El formato del cliente es `python3 cliente.py <host> <puerto>` con la bandera opcional `--debug`, y el archivo se pasa por entrada estándar. El formato del servidor es `python3 servidor.py <host> <puerto>` con las banderas opcionales `--debug` y `--buff N` para fijar los bytes que `recv` entrega por llamada. Sin `--debug` ambos funcionan igual pero no imprimen las líneas de la clase.

Repositorio de GitHub

El código de la actividad está disponible en el repositorio de GitHub al que el equipo docente tiene acceso. El enlace al repositorio es el siguiente.

[https://github.com/julianticucho/cc4303_tareas]

La rama de la entrega es control2parte1.

Declaración de uso de IA

Se utilizó Gemini de Google como apoyo principalmente para consultar dudas sobre Python, como el manejo de binarios y bytes, la construcción y el parseo de headers y la lógica de retransmisión con temporizadores. También se usó el autocompletado que viene por defecto en VSCode. El código, las decisiones de diseño y las pruebas fueron escritas y validadas de forma manual para asegurar su correcto funcionamiento.